

Verilog Implementation of a Cache-Coherent Multiprocessor

Roshan Sundaram *Duke University*
Durham, NC
email@duke.edu



Abstract—As physical constraints limit the frequency scaling of single-core processors, modern compute performance relies heavily on multi-core architectures and shared-memory paradigms. However, maintaining cache coherence across multiple cores introduces significant microarchitectural complexity. This paper details the design, implementation, and testing of a dual-core, cache-coherent multiprocessor implemented in Verilog. Built around the auto-generated WARP-V CPU core, the architecture features custom first-level data caches (L1D), a shared second-level (L2) cache, and a bus arbiter to manage transactions. Coherence is maintained via a hardware MSI snooping protocol. The system is validated using a suite of custom assembly microbenchmarks designed to stress true sharing, false sharing, and capacity evictions. This project bridges the gap between high-level architectural theory and low-level hardware implementation by designing a fully coherent, dual-core RISC-V processor in Verilog.

1 INTRODUCTION

The widespread adoption of multicore processors within the computing industry has been driven by multiple factors. First, hardware designers encountered strict power constraints that limited their ability to further increase processor frequency without permanently damaging the physical hardware. Second, Moore’s Law (along with rapidly-advancing fabrication techniques) has allowed for year-on-year increases in transistor count. Taken together, these two factors led to the industry pivoting to chip multiprocessors in the early 2000s, with companies like Intel and AMD starting to offer consumer products with multiple processing cores on a single die.

The challenge with any system involving multiple independent processing cores, whether it be a chip multiprocessor or one with many separate nodes, is ensuring that all cores have a consistency view of memory while simultaneously uti-

lizing caching to improve processor performance. However, connecting multiple cores together and exploiting thread-level parallelism relies on the fundamental assumption that the memory system will behave in a predictable, predefined manner. This requirement is known as memory consistency, and is concerned with the global ordering of load and store instructions. Consistency, being an architectural specification, is visible to the programmer. *Microarchitects* may implement various microarchitectural features to guarantee consistency, however. To that end, cache coherence protocols are systems that support consistency models but are invisible to the programmer. Various coherence protocols – or no coherence at all – may be used to uphold a consistency model.

In this paper, we present the design and implementation of a dual-core, cache-coherent multiprocessor in Verilog. In Section ??, we provide background on cache coherence protocols and the RISC-V ISA. In Section ??, we discuss the academic and technical merits of embarking on such a project. Then, in Section ??, we introduce the coherence protocol used in our design, and in Section ??, we examine the specific hardware decisions made to implement the protocol. Finally, in Sections ?? and ??, we discuss the verification and testing of the system, and in Section ??, we summarize our work and discuss future directions.

2 BACKGROUND

In each subsequent computer architecture course, students are introduced to more complex and high-performance processor designs. In introductory architectures – ECE250 – we learn about the basic components of a processor and build a simple

single-cycle core in Logisim. In digital design – ECE350 – we expand our understanding of hardware design and implement a five-stage, pipelined processor in Verilog. In advanced architectures I – ECE552 – we study techniques for performance improvement, such as out-of-order execution, branch prediction, and superscalar execution, and use simulators like GEM5 to evaluate the performance of these techniques. Finally, in advanced architectures II – ECE652 – we cover a wide range of shared-memory multiprocessor designs and parallel architectures.

Of particular interest within the topics covered in ECE652 is cache coherence. As we will discuss in Section ??, cache coherence protocols are systems that maintain a consistent view of memory across multiple cores while still allowing for caching to improve performance. Coherence protocols are invisible to the programmer, but are critical for ensuring that the system behaves in a predictable manner. However, the complexity of coherence protocols can be difficult to grasp without hands-on experience. By building a cache-coherent multiprocessor in a hardware description language like Verilog, we can gain a deeper understanding of how coherence protocols work and the challenges involved in implementing them.

3 MOTIVATION

The allure of designing hardware has been ever-present in my academic career. In ECE350, we learned how to use Verilog to realize our own circuit designs, and demonstrated that understanding by building a pipelined processor. However, the processor we built as a single-core design, had a primitive memory system, and a very limited subset of the MIPS instruction set architecture (ISA). Therefore, a natural extension of that project would be to build a more complex multi-core design that supports a more modern and widely-used ISA and a shared memory system.

To that end, RISC-V was chosen as the target ISA for this project. RISC-V is an open-source ISA that has gained significant traction in industry and academia for its ease of use, strong ecosystem of developers and tools, and similarities to ISAs used in education (such as MIPS). Due to RISC-V's similarities to MIPS, the learning curve for implementing a RISC-V core is not steep, as there are many resources available for understanding the ISA and how to implement it. Choosing RISC-V also allows us to leverage a vast array of open-source tools like

core generators, assemblers, and compilers, which can significantly speed up the development process.

Experience with Verilog is also a motivating factor for this project. Although our project in ECE350 was undertaken in Verilog, one semester of formal training is insufficient for understanding the nuances and capabilities of a hardware description language. By building a more complex design, we can gain a deeper understanding of how to use Verilog effectively, and how to design and implement hardware systems in general.

Finally, the project is motivated by the desire to close the gap between theory and reality. In class, we study various cache coherence protocols and shared-memory multiprocessor designs, often with the assistance of state transition diagrams and other visual aids, but we do not have the opportunity to implement these concepts in hardware. By developing a coherence protocol in Verilog, we can gain a deeper understanding of how these concepts work in practice, and the challenges involved in implementing them.

4 COHERENCE

5 HARDWARE ARCHITECTURE

6 VERIFICATION AND DEBUGGING

7 MICROBENCHMARKS AND EVALUATION

8 CONCLUSION AND FUTURE WORK

REFERENCES